KONRAD LORENZ
FUNDACIÓN UNIVERSITARIA

Desarrollo de Aplicaciones Móviles con React Native

René Lobo
2025-II

Requisitos previos

- Node.js (LTS) y NPM
- https://nodejs.org/en/download/package-manager
- Editor de código (Visual Studio Code, Webstorm)
- Git para control de versiones

- **Siempre para empezar desde 0**
  npx create-expo-app --template blank
- **Para acceso desde web**
- npx expo install expo-router
- npx expo install react-native-web react-dom

- cd nombre_carpeta_app
- npx expo start

EXPLORADOR

P12025

- assets
- node_modules
- .gitignore
- App.js
- app.json
- index.js
- package-lock.json
- package.json

App.js > ...

```
1  import { StatusBar } from 'expo-status-bar';
2  import { StyleSheet, Text, View } from 'react-native';
3
4  export default function App() {
5    return (
6      <View style={styles.container}>
7        <Text>Open up App.js to start working on your app!</Text>
8        <StatusBar style="auto" />
9      </View>
10   );
11 }
12
13 const styles = StyleSheet.create({
14   container: {
15     flex: 1,
16     backgroundColor: '#fff',
17     alignItems: 'center',
18     justifyContent: 'center',
19   },
20 });
21
```

¿Pero si no tenemos acceso?

- Usaremos
- https://snack.expo.dev/

Introducción a los componentes en React Native

- React Native proporciona un conjunto de componentes básicos predefinidos que son esenciales para construir interfaces de usuario. Entre los más comunes están: View: Es un contenedor fundamental, similar a un en HTML. Se usa para agrupar otros componentes y aplicar estilos.
- Text: Se utiliza para mostrar texto en la aplicación.
- Image: Para mostrar imágenes.
- ScrollView: Un contenedor que permite desplazamiento.
- TouchableOpacity: Para crear elementos tocables con efecto de opacidad.
- TextInput: Para entradas de texto del usuario.

Componente View

- El componente más fundamental para construir una interfaz de usuario, View es un contenedor que soporta diseño con flexbox, estilos, manejo de algunos eventos táctiles y controles de accesibilidad. View se mapea directamente al equivalente de vista nativa en cualquier plataforma en la que se esté ejecutando React Native, ya sea un UIView, , android.view, etc.
- View está diseñado para ser anidado dentro de otras vistas y puede tener de 0 a muchos hijos de cualquier tipo.

- https://reactnative.dev/docs/view

Componente Text

- Text para mostrar texto en la interfaz
- Soporta anidación, estilos, manejo de algunos eventos táctiles
- https://reactnative.dev/docs/text

Componente Image

- Un componente de React para mostrar diferentes tipos de imágenes, incluyendo imágenes de red, recursos estáticos, imágenes temporales locales e imágenes desde el disco local, como el carrete de la cámara.
- https://reactnative.dev/docs/image

Componente Button

- Un componente de botón básico que debería renderizarse bien en cualquier plataforma. Soporta un nivel mínimo de personalización.
- https://reactnative.dev/docs/button

Componentes adicionales

- Otros componentes útiles en React Native (FlatList, ScrollView, TextInput, TouchableOpacity)

Introducción a los estilos en React Native

- Todos los componentes principales aceptan una propiedad llamada style. Los nombres y valores de estilo generalmente coinciden con cómo funciona CSS en la web, excepto que los nombres están escritos en camel case, por ejemplo, backgroundColor en lugar de background-color.
- https://reactnative.dev/docs/style

StyleSheet

- A medida que un componente se vuelve más complejo, a menudo es más limpio usar StyleSheet.create para definir varios estilos en un solo lugar.
- Código: Ejemplo de creación y uso de una hoja de estilo con StyleSheet
- Ventajas de usar StyleSheet: reutilización, legibilidad

Flexbox para diseño y layout

- Un componente puede especificar la disposición de sus elementos secundarios utilizando el algoritmo Flexbox. Flexbox está diseñado para proporcionar un diseño coherente en diferentes tamaños de pantalla.
- Normalmente, utilizarás una combinación de flexDirection, alignItems y justifyContent para lograr la disposición adecuada.
- Propiedades comunes de Flexbox (flex, flexDirection, justifyContent, alignItems)
- https://reactnative.dev/docs/flexbox

Estilos condicionales y dinámicos

- Aplicación de estilos basados en condiciones o estados
- Código: Ejemplo de estilos condicionales utilizando el operador ternario
- Estilos dinámicos utilizando variables o props

- A partir de acá todos los ejemplos están hechos para que funcionen con solo copiar y pegar en snack

```
import { Text, SafeAreaView, StyleSheet,View,TouchableOpacity } from 'react-native';
import React, { useState } from 'react';
// You can import supported modules from npm
import { Card } from 'react-native-paper';

// or any files within the Snack
import AssetExample from './components/AssetExample';

export default function App() {
const estaActivo = false;
return (
<View>
<Text style={[
styles.texto,
estaActivo && styles.textoActivo
]}>
Este texto tiene estilo condicional
</Text>
<Text style={[
styles.texto,
estaActivo ? styles.textoActivo : styles.textoInactivo
]}>
Este texto alterna entre dos estilos
</Text>
</View>
);
}

const styles = StyleSheet.create({
texto: {
fontSize: 18,
marginBottom: 10,
},
textoActivo: {
color: 'green',
fontWeight: 'bold',
},
textoInactivo: {
color: 'gray',
fontStyle: 'italic',
},
});
```

```
import { Text, SafeAreaView, StyleSheet,View,TouchableOpacity } from 'react-native';
import React, { useState } from 'react';
// You can import supported modules from npm
import { Card } from 'react-native-paper';

// or any files within the Snack
import AssetExample from './components/AssetExample';

export default function App() {
const [estaActivo, setEstaActivo] = useState(false);
return (
<View>
<Text style={[
styles.texto,
estaActivo && styles.textoActivo
]}>
Este texto cambia de estilo
</Text>
<TouchableOpacity onPress={() => setEstaActivo(!estaActivo)}>
<Text>Cambiar estado</Text>
</TouchableOpacity>
</View>
);
}

const styles = StyleSheet.create({
texto: {
fontSize: 18,
color: 'black',
},
textoActivo: {
color: 'red',
fontWeight: 'bold',
},
});
```

¡Vamos a practicar!

- Siguiendo el código base de https://snack.expo.dev/@raloboq/7d156f
- O de esta
- https://snack.expo.dev/@raloboq/miappformula1

- Desarrollar una aplicación móvil de tipo estática que muestre información sobre su deporte favorito

- Agreguen mas texto, imágenes, secciones lo que se les ocurra :D

- Esta actividad se puede hacer individual o de a parejas
- **Se debe realizar la entrega en la semana 5**

Introducción a los hooks en React

Los hooks son funciones especiales que te permiten "engancharte" a características de React en componentes funcionales. Algunos hooks comunes incluyen:

1. useState: Para añadir estado local a componentes funcionales.
2. useEffect: Para realizar efectos secundarios (como suscripciones o cambios en el DOM).
3. useContext: Para consumir un contexto de React.
4. useRef: Para mantener una referencia mutable.
5. useMemo y useCallback: Para optimizaciones de rendimiento.

Ventajas de usar hooks (reutilización de lógica, código más conciso)

Hook useState

- useState es un hook que te permite añadir estado a los componentes funcionales. Básicamente, te permite tener variables que, al cambiar, provocan que el componente se vuelva a renderizar.
- Estructura básica de useState:
- const [variable, setVariable] = useState(valorInicial);
- variable es el valor actual del estado. setVariable es una función que usas para actualizar el estado. valorInicial es el valor con el que se inicializa el estado.

Ejemplo 1

```
import React, { useState } from 'react';
import { View, Text, Button } from 'react-native';


export default function App() {
const [contador, setContador] = useState(0);

return (
<View>
<Text>Contador: {contador}</Text>
<Button title="Incrementar" onPress={() => setContador(contador + 1)} />
<Button title="Decrementar" onPress={() => setContador(contador - 1)} />
</View>
);

}
```

Ejemplo 2

```
import React, { useState } from 'react';
import { View, TextInput, Text, Button } from 'react-native';

export default function App() {
const [nombre, setNombre] = useState('');
const [edad, setEdad] = useState('');

return (
<View>
<TextInput
placeholder="Nombre"
value={nombre}
onChangeText={setNombre}
/>
<TextInput
placeholder="Edad"
value={edad}
onChangeText={setEdad}
keyboardType="numeric"
/>
<Button
title="Enviar"
onPress={() => alert(`Nombre: ${nombre}, Edad: ${edad}`)}
/>
<Text>Nombre: {nombre}, Edad: {edad}</Text>
</View>
);
}
```

Ejemplo 3

```
import React, { useState } from 'react';
import { View, Text, Switch } from 'react-native';

export default function App() {
const [estaActivo, setEstaActivo] = useState(false);
return (
<View>
<Switch
value={estaActivo}
onValueChange={setEstaActivo}
/>
<Text>El interruptor está: {estaActivo ? 'Encendido' :
'Apagado'}</Text>
</View>
);
}
```

Puntos clave para entender useState:

1. Siempre usa la función "set" (como setContador) para actualizar el estado, nunca modifiques la variable directamente.
2. Cuando el estado cambia, el componente se vuelve a renderizar.
3. El valor inicial que pasas a useState solo se usa en la primera renderización.
4. Puedes tener múltiples estados en un componente.

Hook useEffect

- useEffect es un hook que te permite realizar efectos secundarios en componentes funcionales. Estos efectos secundarios pueden ser cosas como suscripciones a datos o fetching de datos.
- Estructura básica de useEffect:

```
useEffect(() => {
  // Código del efecto
  return () => {
    // Código de limpieza (opcional)
  };
}, [dependencias]);
```

Ejemplo 1: efecto que se ejecuta después de cada renderizado:

```
import { View, Text, Button } from 'react-native';
import React, { useState, useEffect } from 'react';


export default function App() {
const [contador, setContador] = useState(0);

useEffect(() => {
alert(`El contador se actualizó a: ${contador}`);
});

return (
<View>
<Text>Contador: {contador}</Text>
<Button title="Incrementar" onPress={() => setContador(contador + 1)}
/>
</View>
);
}
```

En este ejemplo, el efecto se ejecuta después de cada renderizado, imprimiendo el valor actual del contador.

Ejemplo 2: Efecto que se ejecuta solo una vez

```
import { View, Text, Button } from 'react-native';
import React, { useState, useEffect } from 'react';


export default function App() {
const [datos, setDatos] = useState(null);

useEffect(() => {
const fetchDatos = async () => {
try {
const respuesta = await fetch('https://api.ejemplo.com/datos');
const datosJSON = await respuesta.json();
setDatos(datosJSON);
} catch (error) {
console.error("Error al obtener datos:", error);
}
};

fetchDatos();
}, []); // Array de dependencias vacío

return (
<View>
<Text>{datos ? JSON.stringify(datos) : 'Cargando...'}</Text>
</View>
);
}
```

Aquí, el efecto se ejecuta solo una vez cuando el componente se monta, debido al array de dependencias vacío [].

Ejemplo 3: Efecto con limpieza

```
import { View, Text, Button } from 'react-native';
import React, { useState, useEffect } from 'react';


export default function App() {
const [segundos, setSegundos] = useState(0);

useEffect(() => {
const intervalId = setInterval(() => {
setSegundos(prevSegundos => prevSegundos + 1);
}, 1000);

// Función de limpieza
return () => {
clearInterval(intervalId);
};
}, []); // Array de dependencias vacío

return (
<View>
<Text>Segundos transcurridos: {segundos}</Text>
</View>
);
}
```

En este ejemplo, el efecto configura un intervalo y proporciona una función de limpieza para eliminarlo cuando el componente se desmonta.

Ejemplo 4: Efecto que se ejecuta cuando cambian ciertas variables

```
import { View, Text, TextInput } from 'react-native';
import React, { useState, useEffect } from 'react';

export default function App() {
const [busqueda, setBusqueda] = useState('');
const [resultados, setResultados] = useState([]);

useEffect(() => {
const buscarUsuarios = async () => {
if (busqueda.length > 2) {
try {
const respuesta = await fetch(`https://api.ejemplo.com/usuarios?q=${busqueda}`);
const usuarios = await respuesta.json();
setResultados(usuarios);
} catch (error) {
console.error("Error en la búsqueda:", error);
}
} else {
setResultados([]);
}
};

buscarUsuarios();
}, [busqueda]); // Array de dependencias con 'busqueda'

return (
<View>
<TextInput
value={busqueda}
onChangeText={setBusqueda}
placeholder="Buscar usuarios"
/>
<Text>Resultados: {resultados.length}</Text>
</View>
);
}
```

En este caso, el efecto se ejecuta cada vez que cambia el valor de busqueda.

Otros hooks útiles

- Hook **useContext** para acceder al contexto de React. Se usa para manejar el estado global de una aplicación.
- Es útil para manejar estados globales como temas, autenticación o preferencias de usuario.
- Hook useReducer para manejo de estado más complejo. Es especialmente útil cuando tienes múltiples acciones que pueden modificar el estado.